FinEvo: Self-Generated Challenging Tasks for Recursive Self-Improvement of Financial Spreadsheet Agents

Anonymous ACL submission

Abstract

Recent advances in the reasoning and tool-use capabilities of large language models have opened new avenues for recursive self-improvement (RSI), in which models generate their own learning experiences to improve their capabilities. However, enabling the same model to autonomously construct training tasks and improve through its own interactions and learning, without solution guidance from a stronger model, remains a key challenge. We introduce FinEvo, a same-model RSI framework for financial spreadsheet agents that connects source-model preparation, feedback-driven task generation and reinforcement learning. Starting from complete financial models, the framework uses the same model in separate contexts to clean and verify workbooks, evolve challenging modeling and debugging tasks through solver feedback, and learn from its own interactions and executable rewards. On 110 tasks constructed from 11 prepared financial workbooks, we compare the untrained and trained policies on held-out development tasks: task success increases from 22.73% to 29.09%, and target accuracy from 52.52% to 58.25%. These results demonstrate the training value of self-generated tasks and establish an evaluated cycle from same-model autonomous task generation through reinforcement learning to verification of self-improvement.

1 Introduction

Recent advances in large language model (LLM) reasoning, tool use and environment interaction are turning the possibility of AI systems contributing to their own improvement into a concrete research question. Recursive self-improvement (RSI) traces its intellectual roots to machines that design better successors and self-referential systems that modify their own programs (Good, 1965; Schmidhuber, 2003). Its central concern is feeding a system's experience, feedback and improvements back into its development, so that existing capabilities become a basis for subsequent improvement. The target may be model parameters, agent tools, workflows or learning mechanisms.

Recent work explores this direction at different levels. STOP and the Darwin Gödel Machine study agents improving their own programs or workflows (Zelikman et al., 2024; Zhang et al., 2025). Another line concerns autonomous generation of learning experience, allowing models to help determine what they learn next. Socratic learning emphasizes that agent outputs can become subsequent learning inputs, with informative feedback, experience coverage and resources determining whether improvement is possible (Schaul, 2024). Absolute Zero jointly trains task proposal and solving within one model, while R-Zero evolves a curriculum through challenger–solver interaction (Zhao et al., 2025; Huang et al., 2025). These approaches raise an important question: how can such mechanisms extend to agent tasks involving complex artifacts, domain assumptions and multistep tool use?

Financial spreadsheets provide a practically valuable environment for this question. Workbooks organize business assumptions, formulas and cross-sheet dependencies into executable models. Agents must understand these relationships and use inspection, editing and recalculation to complete modeling or debugging tasks. Complete workbooks also provide reusable task sources: their business relationships support tasks that require reconstructing calculations or repairing errors, with submissions checked against reference results. Financial spreadsheets can therefore serve both as an application domain and as an environment for models to construct training experience, obtain feedback and learn.

Two connected challenges arise. *First, tasks*

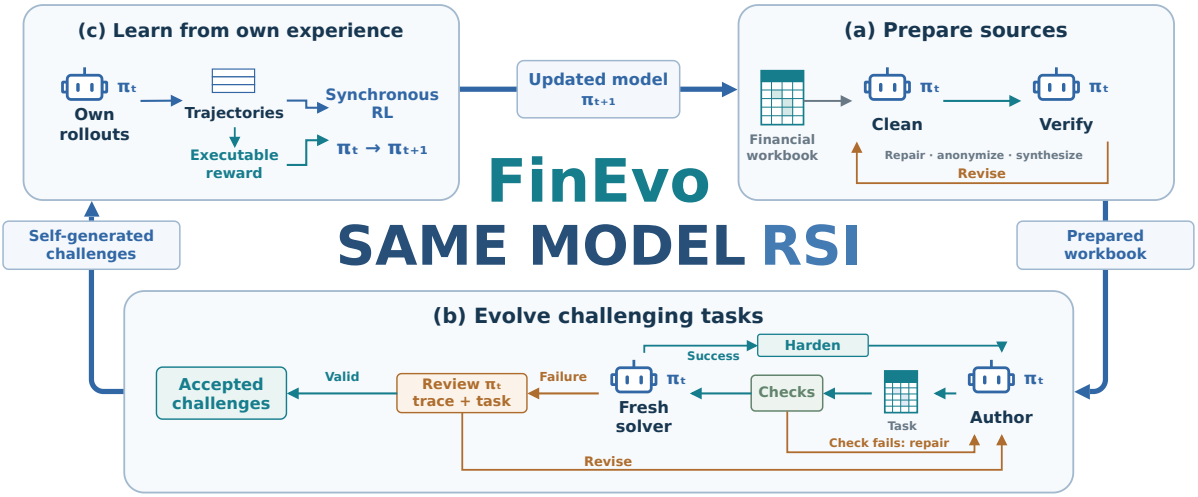


Figure 1: The same-model RSI framework of FinEvo. (c) The model learns from its own rollouts and executable rewards. (a) Separate cleaner and auditor contexts prepare source workbooks through verification and revision. (b) The shared model constructs and evolves challenges using fresh solver feedback. The return arrow depicts recursive reuse of the improved model; reported experiments evaluate one generation–learning cycle on fixed prepared sources.

must be challenging while retaining a clear basis for solving them. This starts with the source model: inconsistent business definitions, broken dependencies or missing assumptions can propagate into every derived task. Even with a prepared workbook, removing more formulas or injecting more errors can destroy necessary evidence rather than create useful challenges. Source verification and solver-guided evolution must therefore work together. *Second, generated challenges must yield useful learning signals.* A failed solve records an unsuccessful interaction; the training value of self-generated tasks must be tested through parameter learning and held-out evaluation. Connecting these questions moves autonomous task generation toward evaluated self-improvement.

To this end, we introduce FinEvo, a same-model RSI framework connecting financial-model preparation, autonomous challenge evolution and RL (Figure 1). The same model first acts as cleaner and auditor in separate contexts, repairing source workbooks and checking their business consistency before publication. Only the prepared workbook is passed to the task author. The author constructs modeling and debugging tasks, and a fresh same-model solver attempts them. Success triggers analysis of the solution and a harder successor; failure opens review for task repair or acceptance as a valid challenge. Fixed program checks and executable scoring support this evolution without replacing business-level review.

During learning, the same target model collects interactions on its generated tasks and receives executable outcome rewards for RL, without stronger-model task solutions or training demonstrations. Strictly synchronized training uses fresh interactions from the current policy at every update, while held-out development tasks track learning performance. The framework connects source preparation, autonomous task supply, training experience and parameter updates; generation, learning and improvement assessment form the evaluated self-improvement cycle reported here.

We use Qwen3.6-27B as the common target model for authoring, solving and RL. From 11 financial workbooks, we construct 110 modeling and debugging tasks and retain complete candidate histories to analyze how feedback changes task difficulty, public evidence and revision direction. Longitudinal cases show that hardening can change the required inference without adding targets, while repair can improve public evidence even when solver reward stays unchanged. From the pretrained policy to update eight, task success on within-source held-out development tasks increases from 22.73% to 29.09%, and target accuracy from 52.52% to 58.25%, with whole-task success gains in both task types. These results characterize autonomous task evolution and its training value, supporting the integration of task generation, learning and improvement assessment.

Our contributions are threefold.

(1) Same-model autonomous task generation. We introduce a pipeline that uses one model for source cleaning and verification, followed by solver-feedback-driven task evolution through success-driven hardening and failure attribution and repair. Complete task lineages support analysis of difficulty adjustment and task-quality changes.

(2) RL learning from self-generated tasks. We train with the target model's own interactions and executable rewards, evaluating how self-generated tasks support capability improvement through overall learning trajectories and task-type breakdowns.

(3) A same-model self-improvement loop. We connect autonomous task generation, RL and held-out evaluation, completing and systematically evaluating the full cycle from same-model task supply and parameter learning to verification of self-improvement.

2 Related Work

Self-generated tasks and RSI. Research on self-improvement studies both changes to agent programs and learning from model-generated experience. STOP uses an LLM-based program improver to refine its own scaffolding, while the Darwin Gödel Machine evolves agent code and empirically validates modifications on coding benchmarks (Zelikman et al., 2024; Zhang et al., 2025). For parameter learning, Absolute Zero trains a unified task proposer and solver, using code execution to validate self-proposed reasoning tasks and provide learning feedback (Zhao et al., 2025). R-Zero initializes separate challenger and solver models from the same base, rewarding the challenger for tasks near the capability boundary of the solver and training the solver on the resulting curriculum (Huang et al., 2025). CURE jointly optimizes coding and unit-test generation in one model using rewards derived from execution outcomes (Wang et al., 2025). It removes the need for ground-truth code solutions during training, while retaining ground-truth unit tests to assess code correctness.

Generating tasks from existing artifacts. Existing codebases and completed workbooks provide reference artifacts for constructing agent tasks. SWE-smith creates software-repair tasks by introducing bugs into real repositories through model-generated edits, procedural modifications, combinations of bugs and reversed pull-request changes (Yang et al., 2025). Candidates are retained when they break previously passing tests; generated issue descriptions and successful repair trajectories support agent training. Workbook Time Machine constructs spreadsheet-creation tasks by removing derived artifacts from completed workbooks and generating requests to reconstruct them (Uniyal et al., 2026). Its dependency-aware edit graph enumerates candidate construction histories over formulas, charts, pivot tables and conditional formatting. Input–output workbook pairs are then sampled from these histories, with instructions generated at different levels of specificity.

Spreadsheet tasks and agent training. SpreadsheetBench focuses on operation-centric requests collected from Excel forums, including extraction, lookup, aggregation and formatting, with multiple test cases used to assess solution robustness (Ma et al., 2024). SpreadsheetBench 2 shifts evaluation toward end-to-end business workflows, including financial modeling, debugging and visualization over interdependent worksheets (Zhu et al., 2026). Experts construct tasks from complete reference workbooks by removing content or injecting errors; financial modeling requires recovering cross-sheet calculations, while debugging requires locating faults without explicit error positions. Spreadsheet-RL extends spreadsheet research to agent training, automatically constructing initial–final workbook pairs from forum discussions with strong coding agents and optimizing multi-turn policies through outcome-based GRPO in Microsoft Excel (Chi et al., 2026). Its Domain-Spreadsheet evaluation further covers finance and other professional domains.

3 FinEvo

3.1 Financial Models as Source Workbooks

A financial model is an executable representation of business relationships in a spreadsheet workbook. Input assumptions and historical data drive formulas for operating forecasts, linked financial statements, valuation, financing or contract schedules. For example, revenue and a cost-rate assumption determine operating costs, which propagate into profit and cash flow. Worksheets encode these relationships through formulas, labels, units,

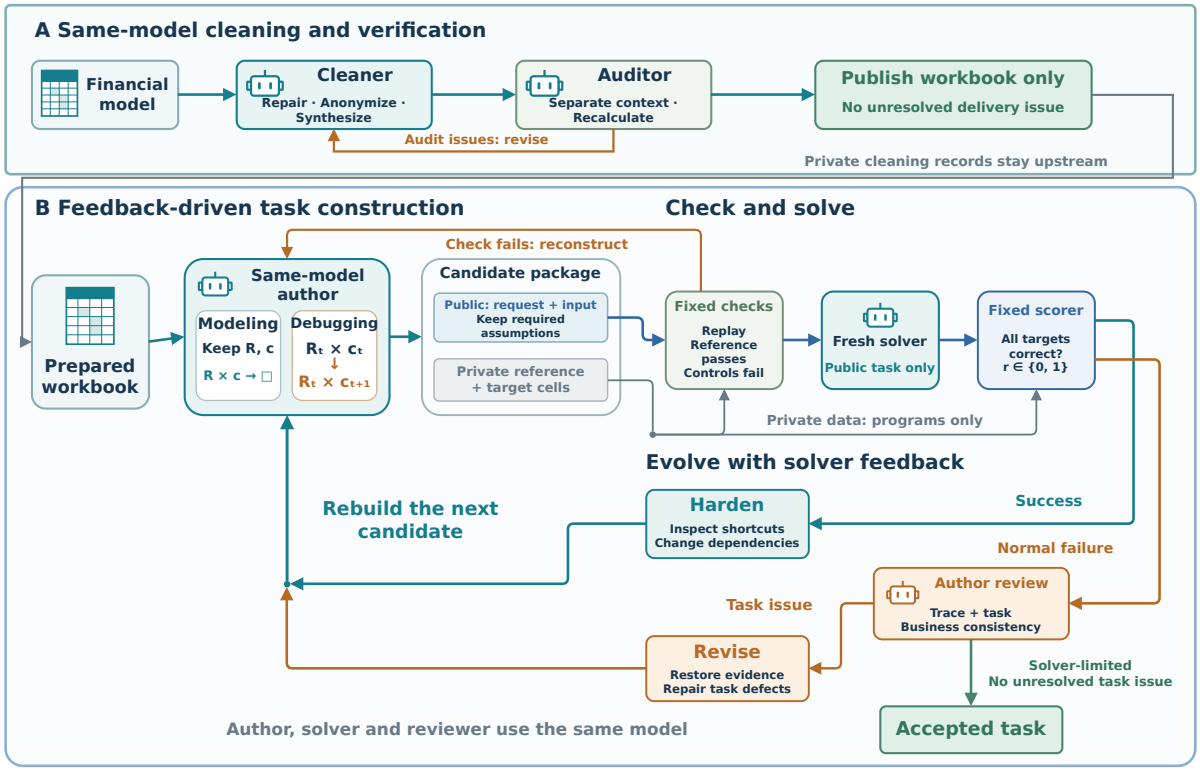


Figure 2: From financial models to training challenges. (A) The same model cleans a source workbook and audits it in a separate context, returning unresolved issues for revision. (B) The prepared workbook seeds feedback-driven task construction. Private references feed fixed programs; the fresh solver receives only the public task. Success triggers hardening; normal failure opens review for repair or qualified acceptance. Content changes return to construction and checks.

dates and notes. The pipeline takes complete models and any accompanying source materials, rather than isolated spreadsheet questions.

Complete models provide both calculation structure and reference results for task construction. Required assumptions must remain distinguishable from quantities to be inferred: a formula may carry an independent assumption, while a hardcoded value may be a derived result. This distinction guides both cleaning and subsequent task transformations.

3.2 Same-Model Cleaning and Verification

The upper flow in Figure 2 prepares a financial model for downstream task construction: a cleaner produces a revised workbook, an auditor checks it in a separate context, and unresolved issues return for revision. Both roles use the same target model and maintain their own persistent contexts. The cleaner first identifies business definitions, units, periods, formula dependencies and accounting relationships in the source materials. It repairs calculation errors, conflicting definitions and missing assumptions while preserving supported business choices.

The cleaner's three operations in the figure are repair, anonymization and value synthesis. Anonymization replaces real company, project, institution, person and contract names, including residual identifiers in comments, document properties, hidden sheets and external links. Value synthesis creates fictional business values from base inputs and business drivers, then recalculates dependent forecasts and outputs; independent cell perturbations or uniform scaling do not suffice. Replayable code records these operations, with repair decisions and their evidence distinguished from anonymization and synthetic-data transformations. Necessary business definitions remain in ordinary assumption sheets, headers, notes or comments.

The auditor then independently inspects the resulting workbook before consulting source materials and internal records to verify repairs and transformations. Recalculation and independent computations check financial relationships, units, timing and dependencies. As the return arrow in Fig-

ure 2 indicates, unresolved issues trigger another cleaner–auditor round; budget exhaustion stops preparation. The "Publish workbook only output is reached only with no unresolved delivery issue. This workbook becomes the "Prepared workbook input to task construction below; private cleaning analyses, repair guides and identity mappings stay upstream. These checks provide a documented basis for construction, not an exhaustive guarantee of financial correctness.

3.3 Solver-Guided Challenge Evolution

Candidate construction. Panel B of Figure 2 starts from the prepared workbook delivered by panel A. The same-model author selects business regions and dependencies and writes replayable transformations. Its two branches, Modeling and Debugging, respectively remove derived calculations for reconstruction or inject implementation errors for diagnosis and repair. The resulting Candidate package contains a public request and input workbook, a private reference and a private target set T. Transformations preserve the public assumptions needed for solving; a new business module requires its own assumptions, reference calculation and checks.

Fixed checks and solver feedback. The candidate next follows Fixed checks → Fresh solver → Fixed scorer in Figure 2. Checks require replayed edits to match the target set, the reference to score one, and unchanged input and deliberately wrong outputs to score zero. A failed check follows the upper return arrow to the author for reconstruction. After passing, a fresh same-model solver receives only the public task; the private reference and target list feed the fixed checking and scoring programs, as shown by the separate gray path. Scoring compares values after the same LibreOffice recalculation and normalization, using two decimal places for numbers and normalized dates. For task q and attempt j,

$$a_{qj} = |T_q|^{-1} \sum_{c \in T_q} \mathbf{1}[v_{qj}(c) = v_q^*(c)], \quad (1)$$

$$r_{qj} = \mathbf{1}[a_{qj} = 1]. \quad (2)$$

Here v_q^* denotes reference values. Missing or unreadable workbooks receive zero; equivalent formulas or correct values are accepted. Non-target changes are diagnostic, although they may propagate into target errors. Debugging targets are direct injected-error positions. The scorer and its numerical rule remain fixed.

Hardening, review and acceptance. The two outcome paths leave the fixed scorer. The Success path reaches Harden: the author reads the successful trajectory, identifies shortcuts, and constructs a harder successor through cross-sheet dependencies, interacting faults or less reusable formula patterns. The Normal failure path reaches Author review, which examines the trajectory, output and target differences against public evidence and business relationships. Ambiguity, missing assumptions or reference defects lead to Revise. Only failure attributed to solver error, with business consistency checked and no unresolved task issue, reaches Accepted task. A failed attempt alone therefore does not establish a valid challenge.

The Harden and Revise paths merge into the left-hand return arrow to the author, rebuilding the next candidate before checks and solving repeat. The author retains its task history, while each solver starts fresh. Content changes create versions; execution or API failures instead trigger bounded retries or recovery of the same frozen version.

3.4 Reinforcement Learning on Self-Generated Tasks

Reinforcement learning (RL) updates an agent policy from rewards obtained through environment interactions, with the objective of increasing expected task reward. Group Relative Policy Optimization (GRPO) samples multiple trajectories per task and uses their group-normalized rewards to estimate advantages without training a separate value model (Shao et al., 2024).

In the RL stage of Figure 1C, accepted challenges define the training distribution. We use a GRPO-style group-relative update with the agent-training adaptations documented in the accompanying code. The current policy samples multiple tool-use trajectories per task, inspecting and editing workbooks before submission. The fixed scorer assigns binary whole-task rewards. Within-task group normalization compares these outcomes to produce advantages for policy optimization; reference solutions are used by the scorer, not supplied as demonstrations. Learning therefore uses the target model's own interactions on its generated tasks.

We adopt synchronous RL to avoid policy-lag-induced off-policy mismatch: trajectories generated by an outdated policy may not reflect the behavior of the policy being optimized. Aligning data collection with the current policy makes learning feedback correspond to its present capabilities. Held-out tasks assess the resulting policy with the same scoring contract, using mean whole-task success and macro-averaged target accuracy. Together, autonomous task construction, learning from self-generated interactions and held-out improvement assessment close the minimal RSI loop evaluated in this work. The same model's own challenge generation and executable feedback thus become the basis for improving its capabilities.

4 Experiments

4.1 Experimental Setup

Source workbooks. We use 11 internal financial models as source workbooks.

Generation campaign. Each of 11 workbooks seeds five modeling and five debugging tasks. Qwen3.6-27B (Qwen Team, 2026) with OpenHands SDK 1.47.0 (Wang et al., 2024) performs construction, hardening, isolated solving and review at `xhigh` reasoning. Each call has 3,600 seconds and an initial cap of ten candidate versions.

RL training and evaluation. For RL, one of the five tasks in each source–type group is held out, yielding 88 training and 22 development tasks. Training follows the adapted Modal Dojo/Slime recipe (Modal, 2026; Slime Contributors, 2026). Each update samples 32 distinct training queries with eight fresh trajectories each, using the current policy only and admitting no mixed-version batches. The learning rate is 4×10^{-6}, with no KL or entropy penalty. Each checkpoint is evaluated on all 22 development tasks with five attempts per task, temperature 0.6 and a 1,800-second limit. Missing outputs score zero.

Independent solvability check. We conduct a small-scale supplementary solvability check on eight randomly selected tasks generated by FinEvo. GPT-6/Codex solves the same tasks in a separate harness. This comparison provides no solutions or supervision for task acceptance or RL.

4.2 Self-Generated Training Challenges

This section reports experimental results for same-model iterative task generation. In the repeated cycle of construction, checking and independent solving, successful solves trigger further hardening; failures prompt a review of task validity using the trajectory and output. Tasks with identified issues are revised and solved again, while those whose failures are attributed to solver limitations are accepted after review. Round-wise construction statistics and cases of hardening and repair characterize how this feedback loop operates and the tasks it produces.

4.2.1 Round-by-Round Construction

Figure 3 tracks construction progress across 110 task lineages, initialized as five modeling and five debugging tasks from each of 11 cleaned financial models. Accept denotes valid challenges at the target difficulty; Harden denotes valid but solvable candidates. Each round's valid task count equals cumulative Accept plus current-round Harden.

The first-round results highlight the need for iterative construction. Only 13 candidates are accepted and 14 proceed to hardening, giving 27 valid tasks (24.55%) and a target-difficulty yield of 11.82%. The remaining 83 (75.45%) require further work, including 76 that fail pre-solve checks (Table 6, Appendix A).

Most challenges reach the retention criterion within three rounds. By round three, 95 tasks (86.36%) are valid in their current versions and 80 (72.73%) have been accepted. Valid task count reaches 110 at round nine, when 108 are accepted; all 110 are accepted by round ten.

A minority of tasks account for over half of the construction effort. The 30 lineages requiring more than three rounds represent 27.27% of tasks but consume 185/357 (51.82%) of model-authored versions. These lineages require an average of 6.17 authored versions per task, compared with 2.15 for the 80 tasks accepted within three rounds, a 2.87-fold difference in construction effort.

4.2.2 Success-Driven Hardening

Across the 110 tasks in Figure 4, 65 never trigger hardening, 26 trigger it once and 19 trigger it at least twice, with a maximum of six times. Thus 45 tasks (40.91%) undergo 89 hardening steps. Appendix A reports the successor outcomes and target-count comparisons underlying the following analysis.

Trajectory-guided hardening produces challenges beyond the current solver. The author

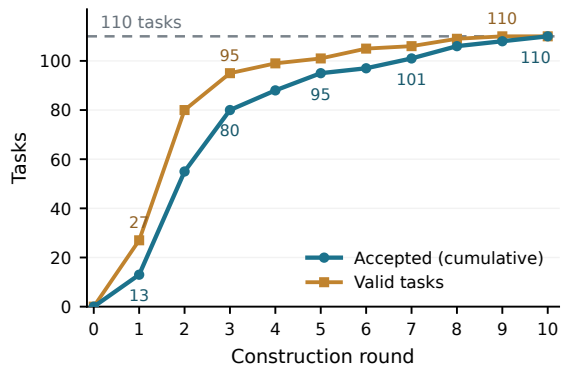


Figure 3: Generation progress across 110 tasks. Accepted is cumulative; Valid equals cumulative Accepted plus current-round Harden. The dashed line marks 110 tasks.

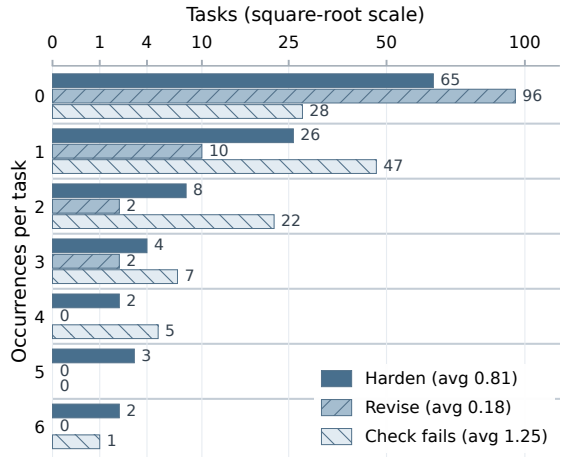


Figure 4: Per-task frequencies of Harden, Revise and Check fails across 110 tasks, with a square-root task-count axis. The legend reports average occurrences per task, including zeros; outcomes follow content-round accounting.

revisits the successful trajectory, script and output to identify formula examples, repeated structures or directly reusable computations, then adjusts the next challenge accordingly; Appendix A documents this feedback protocol. Of 68 immediate hardened successors with normal solver outcomes, 37 (54.41%) are no longer solved successfully, and 33 of these are accepted after author review (Table 2). The remaining four failures are revised. These observed transitions show how successful solves supply actionable feedback for further challenge construction.

Hardening changes both task size and business-dependency reasoning. Among 67 comparable before/after pairs, target count increases in 40, stays unchanged in 23 and decreases in four; eight failed successors have unchanged counts and two have fewer targets (Table 3). The five-target example in Figure 7(a) replaces cash-flow sign/operator faults with driver selection, aggregation scope, period alignment and lease roll-forward errors. Table 8 details all five changes and their original target outcomes: four are corrected, while the next-period CapEx-rate error remains. This case illustrates increased dependency-reasoning demands without increasing target count.

4.2.3 Check- and Review-Driven Task Repair

Figure 4 summarizes the two repair routes: 138 Check fails returns across 82 tasks and 20 review-triggered Revise decisions across 14 tasks. Table 4 records their immediate successor outcomes; Table 5 links the accepted tasks to their preceding transitions.

The two repair routes jointly form a major path to the accepted corpus. Of 110 accepted tasks, 88 (80.00%) encounter at least one Check fails or Revise; eight encounter both. Moreover, 63 tasks (57.27%) are accepted in the immediate successor version after a return: 54 after Check fails and nine after Revise (Table 5). Thus repair participates in most task histories and directly precedes acceptance for over half of the corpus, rather than merely accounting for isolated edits at the end of construction.

Check failures expose unmet verification conditions before independent solving. Of 138 returns, 136 arise during construction/replay checks, one from the unchanged-input negative control and one at the reference gate (Appendix A). In the negative-control case, leaving the input unchanged already earns a passing score, exposing a candidate that does not require the intended repair. Such feedback identifies a concrete validation condition to revisit before interpreting solver performance as task difficulty. These checks therefore support task construction through executable feedback, without relying on a solver failure to reveal the problem.

Failure review uses solver behavior to uncover task issues after checks pass. The 130 normal solver failures lead to 20 Revise decisions, of which 19 cite task issues. The reports include six missing-public-assumption cases, four instruction-scope or meaning cases and three structural-target cases (Appendix A). In F04-D002, review identifies missing public allocation evidence and

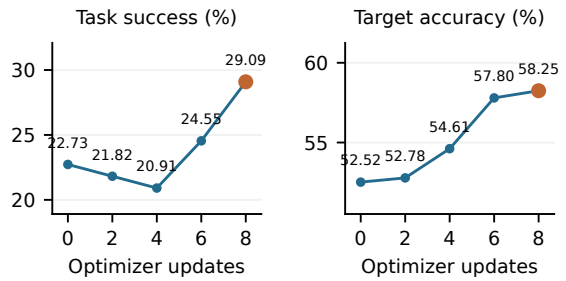


Figure 5: Learning progress before RL and through eight updates: avg5 over 22 tasks with five attempts each. Orange marks the update-eight result.

prompts explicit category labels and ratios to be restored while all 28 targets and their formulas remain unchanged (Figure 7(b), Appendix B). Unlike a check return tied to an unmet validation condition, Revise follows a diagnosis of why attempted solving failed, using the trajectory and output to revisit the task itself.

4.3 RL Self-Improvement

To assess whether the generated tasks can improve the model's own capabilities, we split the 110 tasks constructed above into 88 training tasks and 22 held-out development tasks.

4.3.1 Training Procedure

We perform eight synchronous GRPO-style updates using fresh target-policy interactions on the generated tasks and fixed whole-task rewards. At each development checkpoint, the current policy makes five independent attempts on each of the same 22 tasks. We report avg5 scores by averaging the five attempt scores within each task and then averaging across tasks.

4.3.2 Results and Analysis

Both whole-task success and target accuracy improve over the pretrained reference. From before training to update eight, avg5 task success rises from 22.73% to 29.09%, a gain of 6.36 percentage points or 28.0% relative, while target accuracy rises from 52.52% to 58.25%. Target accuracy increases at every measured checkpoint; task success initially declines before improving at updates six and eight (Figure 5). These metrics capture complementary changes: target accuracy records partial progress at the target level, whereas whole-task success requires every target to be correct.

	Task success (%)		Target accuracy (%)	
Type	Before RL	Step 8	Before RL	Step 8
Modeling	23.64	30.91	54.18	66.83
Debugging	21.82	27.27	50.85	49.66

Table 1: Task-type results before training and after eight updates. Task success is avg5; target accuracy is macro-averaged. Each type has 11 tasks with five attempts each; missing outputs score zero.

Both task types improve in whole-task success, while modeling drives target-accuracy gains. From the pretrained reference to update eight, avg5 task success rises from 23.64% to 30.91% for modeling and from 21.82% to 27.27% for debugging (Table 1). Modeling target accuracy increases from 54.18% to 66.83%, whereas debugging changes from 50.85% to 49.66%. Thus both task types produce more fully correct outputs, but the target-level gain is concentrated in modeling. Each type contains 11 development tasks, evaluated five times per checkpoint.

These are descriptive comparisons on within-source development tasks. For the pretrained-to-update-eight differences, the source-cluster 95% intervals are $[-3.64, 17.27]$ percentage points for task success and $[-3.41, 14.59]$ for target accuracy; both span zero. Retaining the original failures in two retried slots gives a conservative avg5 task success rate of 28.18%.

4.4 Independent Solvability Check

We conduct a small-scale supplementary validation on eight randomly selected tasks generated by FinEvo. GPT-6/Codex completes six tasks (6/8), while Qwen/OpenHands completes none (0/8). These six successful independent solutions provide additional evidence that FinEvo can generate tasks that challenge the target configuration while remaining solvable.

5 Conclusion

FinEvo realizes the minimal same-model RSI loop defined in this work: autonomous construction of financial training challenges, RL on the target model's own interactions and held-out assessment of the observed improvement.

Limitations

Task scope. The RSI loop evaluated in this work is limited to financial Excel tasks. Its effectiveness

across a broader range of tasks and environments remains unverified. Further evaluation is needed to determine whether the same task-generation, feedback and learning process generalizes beyond financial spreadsheets.

Model scale. Our experiments use Qwen3.6-27B, and the RSI loop has yet to be evaluated with larger-parameter models. Whether the observed task-construction and learning benefits persist as model scale increases remains to be established.

Ethical Considerations and Data Access

The financial models originated in nonpublic materials. The v2 preparation process anonymizes identities and synthesizes business values, but this local draft does not assert redistribution rights or a public benchmark release. No original workbooks, company/provider identities or financial values appear in the anonymous numerical reproduction bundle. Any later task release requires the relevant rights and privacy checks. Complete training reproduction requires authorized model and task access; CPU statistical reproduction is a different level of access.

The deterministic scorer does not certify financial suitability, and self-review can miss semantic defects. Passing tasks should not be used as financial advice. No human participants or new personal annotations were collected. AI assistance supported implementation, analysis, writing and internal simulated review; the researcher remains responsible for the evidence and submission declarations. Internal reviews do not constitute official ACL decisions.

References

Banghao Chi, Yining Xie, Mingyuan Wu, Jingcheng Yang, Jize Jiang, Zhaoheng Li, Shengyi Qian, Minjia Zhang, Klara Nahrstedt, Rui Hou, Xiangjun Fan, and Hanchao Yu. 2026. Spreadsheet-RL: Advancing Large Language Model Agents on Realistic Spreadsheet Tasks via Reinforcement Learning. *Preprint*, arXiv:2605.22642.

Irving John Good. 1965. Speculations concerning the first ultraintelligent machine. In *Advances in Computers*, volume 6, pages 31–88. Academic Press.

Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. 2025. R-Zero: Self-evolving reasoning LLM from zero data. *Preprint*, arXiv:2508.05004.

Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. 2024. SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation. *Preprint*, arXiv:2406.14991.

Modal. 2026. Training multi-turn coding agents. Modal Dojo tutorial. Accessed October 16, 2026; local recipe file hashes retained.

Qwen Team. 2026. Qwen3.6-27B model card. Accessed October 16, 2026.

Tom Schaul. 2024. Boundless socratic learning with language games. *Preprint*, arXiv:2411.16905.

Juergen Schmidhuber. 2003. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *Preprint*, arXiv:cs/0309048.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *Preprint*, arXiv:2402.03300.

Slime Contributors. 2026. Slime: An LLM post-training framework. Source revision 3585d4a7eb1a5c108810238b47c37d3107d0a2ba.

Mansi Uniyal, Agamdeep Singh, Ananya Singha, Priyanshu Gupta, Mukul Singh, Gust Verbruggen, Vu Le, and Sumit Gulwani. 2026. Back to the Future: A workbook time machine for spread sheet creation benchmarks. *Preprint*, arXiv:2608.07873.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, and 5 others. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *Preprint*, arXiv:2407.16741.

Yinjie Wang, Ling Yang, Ye Tian, Ke Shen, and Mengdi Wang. 2025. Co-evolving LLM coder and unit tester via reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 38.

John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. SWE-smith: Scaling data for software engineering agents. *Preprint*, arXiv:2504.21798.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling*.

Immediate successor outcome	Count	% of 68
Solved; further hardening	31	45.59
Failed; accepted after review	33	48.53
Failed; revised after review	4	5.88
Normal solver outcomes	68	100.00
No normal solver outcome	21	–
All hardened successors	89	–

Table 2: Immediate outcomes of 89 hardened versions. Percentages use the 68 normal solver outcomes; the other 21 do not enter this denominator.

Target-count change	Solved	Failed	Total
Increase	14	26	40
Unchanged	15	8	23
Decrease	2	2	4
Total	31	36	67

Table 3: Hardening outcomes by target-count change for 67 comparable parent/child pairs. One of the 68 normal successor trials lacks comparable target counts.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2025. Darwin Gödel machine: Open-ended evolution of self-improving agents. *Preprint*, arXiv:2505.22954.

Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. 2025. Absolute zero: Reinforced self-play reasoning with zero data. *Preprint*, arXiv:2505.03335.

Jian Zhu, Yuzheng Zhang, Zeyao Ma, Bohan Zhang, Armin Schoepf, Daniel Woloch, Peter Yiliu Wang, Guangyu Robert Yang, Samuel Jacob, Siddharth Nagisetty, Abhiram Chundru, Jean Lin, Spencer Mateega, and Jing Zhang. 2026. SpreadsheetBench 2: Evaluating Agents on End-to-End Business Spreadsheet Workflows. *Preprint*, arXiv:2606.29955.

A Process Counts and Review Coding

Trajectory-guided hardening feedback. The archived hardening prompt supplies the prior successful trajectory, script, output and scores to the same-model author. It asks the author to identify exploitable formula examples, repeated structures, independent computation modules or directly reusable relations, and to state the next hardening mechanism before generating a new candidate. It permits changes to business derivations, information distribution, cross-sheet dependencies and boundary conditions, while requiring sufficient public evidence. This documents the feedback supplied to the author; the observed successor outcomes do not isolate its effect against a no-trajectory control.

Check- and review-driven repair transitions. Table 4 counts one immediate content-round successor per return, including repeated events within the same task. Eight tasks encounter both routes. Check fails comprises 136 construction/replay returns, one negative-control rejection and one reference-gate return. These stages are distinct from normal solver failures and do not establish that every returned candidate is semantically unsolvable. The successor counts describe observed revisions, not repair effectiveness against an unrepaired control. Table 5 counts each final accepted task once by the disposition immediately before its accepted content version; initial acceptance is separate. The 88 ever-returned tasks are the union of the 82 Check fails and 14 Revise lineages, with eight shared lineages.

Round-wise candidate outcomes. Table 6 provides the per-round dispositions underlying the cumulative progress in Figure 3.

Failure-review issue coding. Single-analyst coding of the 20 revised reviews assigns one primary reported issue per version: six missing public assumptions, four instruction-scope or meaning issues, three structural-target issues, five reference/build/check or recalculation issues, one claimed budget/scope issue and one solver-attributed revision. For example, reports identify deleted growth assumptions, ambiguous amount-versus-share units and intentional blanks treated as targets. The distribution shows what the author attempted to repair: it often revisited the problem specification or answer contract rather than merely reducing difficulty. These are coded author reports, with repeated versions retained, not independently verified defect rates.

Coding uses each review's summary and root causes. The anonymous coding file binds each decision to a task version and original evidence hash. Mixed cases retain notes; repeated versions remain separate. This single-analyst description of author reports has no inter-rater reliability estimate and does not independently validate the claimed defects or successful repairs. Check fails and Revise are distinguished by their trigger and available evidence, not mutually exclusive defect types: review can also report reference or construction issues.

For example, the budget/scope review asserts

Trigger	Tasks	Events	Next Check fails	Next Harden	Next Revise	Next Accept
Check fails	82	138	38	38	8	54
Revise	14	20	4	6	1	9

Table 4: Immediate outcomes after Check fails or Revise. The four successor-outcome columns sum to Events; Tasks counts distinct lineages within each row.

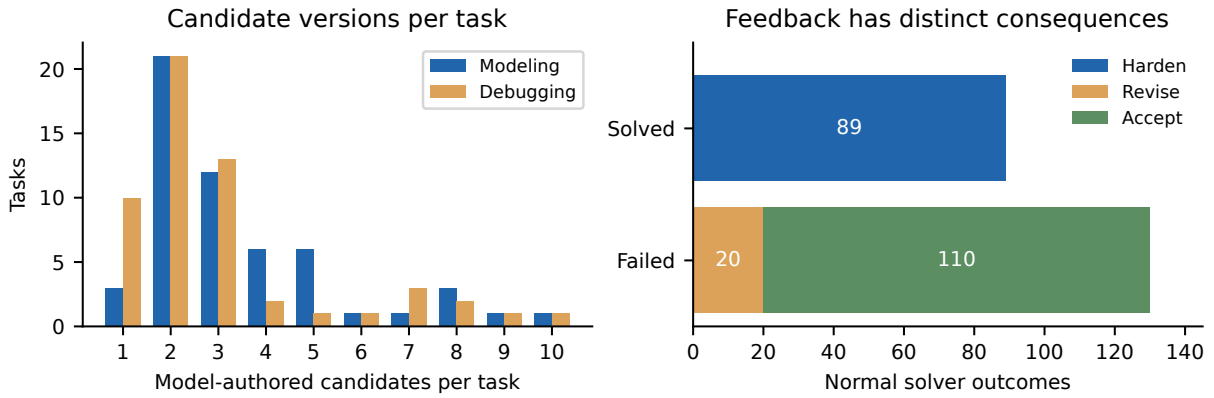


Figure 6: Candidate and feedback counts. Left: model-authored versions per task, excluding one mechanical packaging revision. Right: 219 normal solver outcomes lead to 89 hardenings, 20 review revisions and 110 acceptances. These are selected construction histories, not independent benchmark rates or a generation ablation.

Preceding transition	Accepted tasks
Initial candidate	13
Harden	34
Check fails	54
Revise	9
Total	110

Table 5: Final accepted tasks by preceding transition. Each task appears once. Across complete histories, 88 tasks encounter at least one Check fails or Revise.

Round	In	Accept	Harden	Revise	Check fails	Next
1	110	13	14	7	76	97
2	97	42	25	0	30	55
3	55	25	15	6	9	30
4	30	8	11	3	8	22
5	22	7	6	2	7	15
6	15	2	8	0	5	13
7	13	4	5	1	3	9
8	9	5	3	1	0	4
9	4	2	2	0	0	2
10	2	2	0	0	0	0
Total	357	110	89	20	138	–

Table 6: Final outcomes by model-authored task-content round, mapped to Figure 2B. Check fails follows *Check fails: reconstruct*; Revise follows failure review. Successful solves trigger Harden; acceptance follows failure review. Next equals Harden + Revise + Check fails. Returned candidates undergo construction, checks and solving again; accepted tasks exit. The last row reports cumulative counts.

that several thousand targets are infeasible within an hour, an assertion we do not verify. Another proposes changing recalculation code; this narrative is not evidence of a changed fixed verifier. The separate manifest/hash checks, rather than author claims, establish the final scoring contract. These distinctions prevent reported diagnoses from being silently promoted to ground truth.

For runtime scope, we inspect the retained solver script for every attempt. Of 219 normally classified trials, 192 configure no explicit condenser and 27 configure the same-model 80-event/60,000-token condenser. The 64 abnormal trials include 51 retained scripts without it and 13 missing scripts, whose policy we leave unknown. Final selected trials comprise 97 without and 13 with it. Anonymous task/version/retry rows and runtime signatures are included in the numerical output; scripts are hashed in the private evidence index. A configured condenser does not prove it activated during a given attempt.

Execution accounting and task-quality diagnostics. The candidate counts distinguish 357 model-authored versions from one mechanical packaging version. The 110 tasks constitute the eventual accepted corpus, not an initial-budget yield estimate.

Source	M	D	Reject	Harden	Revise	Missing
F01	11	13	10	4	0	0
F02	18	12	14	5	1	1
F03	14	25	13	14	1	1
F04	13	9	9	1	2	0
F05	11	10	6	3	2	0
F06	20	15	14	11	0	2
F07	22	19	14	16	1	1
F08	23	14	12	11	3	1
F09	16	13	11	2	6	2
F10	25	27	23	16	4	0
F11	18	9	11	6	0	0
Total	191	166	137	89	20	8

Table 7: Anonymous source-level histories. Each source starts five tasks per type. M/D count authored modeling/debugging versions, excluding the mechanical version. Reject counts replay/package rejections; Harden and Revise count transitions; Missing counts final selected attempts without a readable target report. These columns describe different stages and are not additive.

The immediate repair outcomes in Section 4.2.3 count one child per revised review, including repeated revisions within a lineage. They are distinct from the final accepted-task counts.

All 110 accepted packages retain Oracle reward one and both negative-control rewards zero. Their verifier hashes and scoring-contract hashes are unchanged across tasks. A failed hardening attempt therefore cannot be rescued by silently changing the numerical rule. The single unchanged-input pass is rejected as a candidate defect, rather than counted as solver success on a meaningful challenge.

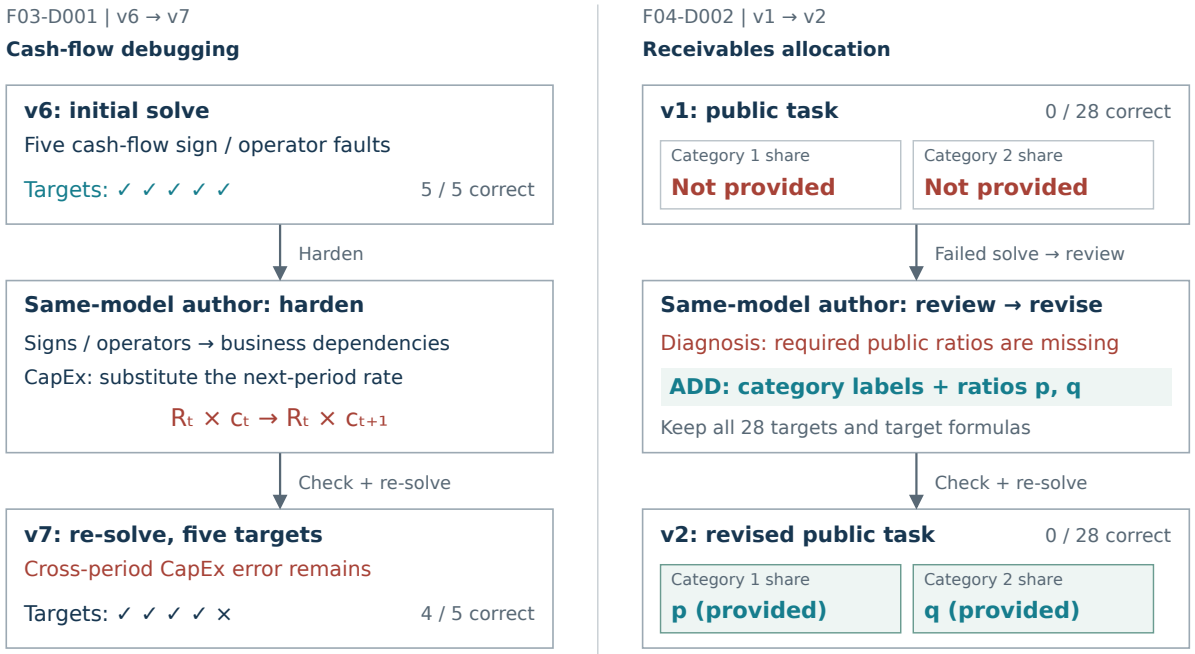
Acceptance still has a limited meaning. The final construction trials include 102 readable target reports and eight missing or unreadable outputs; 14 trials reach the agent time limit. Timeout and submission are overlapping categories. A reviewed zero with no workbook does not establish a financial reasoning failure. The campaign also retains 64 abnormal solver attempts and 22 abnormal Oracle/control attempts. These execution failures are not additional normally failed financial problems. The complete final corpus is pipeline-accepted; it is not a claim that all 110 tasks have been independently certified as unambiguous financial challenges.

B Longitudinal Evidence for Challenge Evolution

These cases are selected to explain distinct design responses, not to estimate their frequency or certify the whole corpus. The case extraction reads original workbooks, candidate records and solver outputs, exports only booleans/counts and binds each file by hash. It performs no LibreOffice run, new scoring or model invocation. Original rewards remain unchanged.

Restore an identifiable allocation. Section 4.2.3 explains the allocation ambiguity and its revision. Direct inspection confirms that v2 adds public category labels and the two reference ratios in an initially empty assumption area; these agree with the reference multipliers. All 28 target positions and their input/reference formulas remain unchanged. Both solver attempts fail all targets, with no timeout and the same configured absence of a condenser. The author revises v1 and accepts v2. These retained checks support the specific public-evidence change, without independently auditing every business relation.

Change the mechanism without adding targets. Table 9 preserves all seven candidates in F03-D001, including the unsuccessful checks. v6 has five cash-flow sign/operator faults. v7 changes the fault locations and families while retaining five targets and exactly the same reference financial-sheet content. The solver scripts are byte-identical and neither configures a condenser. The v7 input uses $\mathrm{CapEx}_t = \mathrm{Revenue}_t \times \mathrm{rate}_{t+1}$ where the reference and adjacent-period pattern use their own current-period rates. Its saved output retains this erroneous formula, and the original report identifies it as the only wrong target. The review's listed five relations correspond to v7. These inspections explain what changed and what the solver missed, but stochastic execution, different target identities and possible unexamined defects prevent a causal estimate of hardening effectiveness.



(a) Success-driven hardening

(b) Failure-driven repair

Figure 7: One iteration of feedback-driven task evolution. (a) Successful solving triggers hardening from sign/operator faults to business-dependency faults. (b) Failure review restores missing public ratios despite unchanged solver scores. Scores indicate correct targets; R and c denote revenue and the CapEx rate.

Business dependency	Reference-to-input change	Target outcome
Receivables driver	$\mathrm{DSO}_t \rightarrow \mathrm{DIO}_t$	Correct
Contract-balance scope	$\Delta(C+N) \rightarrow \Delta C$	Correct
CapEx period	$R_t c_t \rightarrow R_t c_{t+1}$	Missed
Accrued-liability driver	Accrual rate replaced by prepaid rate	Correct
Lease roll-forward	New lease additions omitted	Correct

Table 8: Five business-dependency errors in F03-D001/v7 and their original solver outcomes. The preceding v6 has five sign/operator errors and is fully solved.

In Table 8, DSO/DIO denote sales/inventory days; C and N are current/noncurrent contract balances; R is revenue and c the CapEx rate. The symbolic changes and outcomes come from retained reference/input workbooks and original target reports. They describe this selected case, not corpus-wide frequencies of hardening mechanisms.

Version	Transition	Replay	Targets	Correct
1	Construct	Reject	–	–
2	Construct	Pass	5	5
3	Harden	Pass	7	7
4	Harden	Reject	–	–
5	Construct	Reject	–	–
6	Construct	Pass	5	5
7	Harden	Pass	5	4

Table 9: Complete candidate history for selected lineage F03-D001. Counts describe original normal solver trials; a check rejection has no solver result. Retry attempts are not extra versions.

Design implications.

Check what the constraints identify. Preserving a total does not preserve an identifiable allocation. A correct reference and a balance check can coexist with insufficient evidence for individual categories. Review should trace the public assumptions that support each requested calculation.

Describe the changed inference. Record the business relationships, dependencies and fault mechanisms changed by hardening alongside target count. The equal-count case explains an observed solved-to-failed transition without attribut-

ing it to size. Conversely, the ratio repair improves public evidence without changing the recorded zero reward.

Bind review to the candidate. Business claims in a review should be linked to the frozen candidate's actual targets and dependencies, with that correspondence checked explicitly.

Track learning eligibility separately. Acceptance records a reviewed failed solve; successful demonstrations and reward variation describe other conditions needed by learning algorithms. Maintaining these records separately can identify where additional sampling, different feedback or a different objective may be needed.